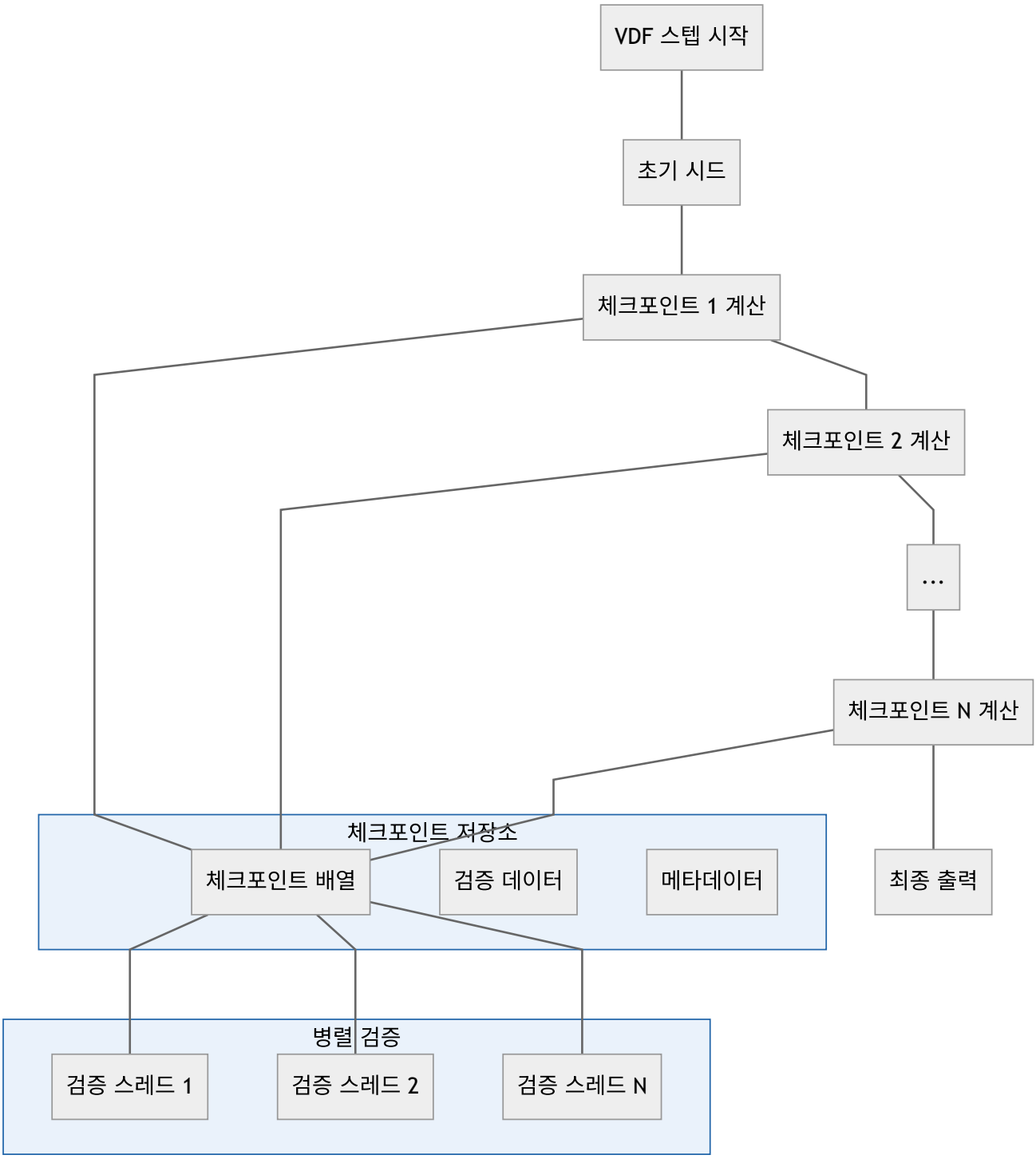
VDF 체크포인트 시스템

개요

VDF 체크포인트 시스템은 Irys 체인에서 VDF 계산의 중간 결과를 저장하고 검증하는 핵심 메커니즘입니다. 이 시스템은 VDF의 순차적 계산을 여러 단계로 나누어 병렬 검증을 가능하게 하고, 네트워크 효율성을 크게 향상시킵니다.

체크포인트 시스템 아키텍처

전체 구조



핵심 구성 요소

1. **체크포인트 생성기**: `vdf_sha` 함수
2. **체크포인트 검증기**: `last_step_checkpoints_is_valid` 함수

3. **병렬 처리 엔진**: Rayon 기반 멀티스레딩

4. **저장소**: 메모리 내 체크포인트 배열

체크포인트 생성 알고리즘 (`vdf_sha`)

함수 시그니처

```
pub fn vdf_sha(
hasher: &mut Sha256, // SHA256 해시 인스턴스
salt: &mut U256, // 솔트 값 (가변)
seed: &mut H256, // 시드 값 (가변)
num_checkpoints: usize, // 체크포인트 개수
num_iterations_per_checkpoint: u64, // 체크포인트당 반복 횟수
checkpoints: &mut [H256], // 결과 저장 배열
)
```

상세 동작 과정

1. 초기화 단계

```
let mut local_salt: [u8; 32] = [0; 32];
```

- 32바이트 솔트 버퍼 생성
- 리틀 엔디안 형식으로 솔트 저장

2. 체크포인트별 계산 루프

```
for checkpoint_idx in 0..num_checkpoints {
// 솔트를 바이트 배열로 변환
salt.to_little_endian(&mut local_salt);
// 지정된 횟수만큼 해시 반복
for _ in 0..num_iterations_per_checkpoint {
hasher.update(local_salt); // 솔트 추가
hasher.update(seed.as_bytes()); // 현재 시드 추가
*seed = H256(hasher.finalize_reset().into()); // 새 시드 생성
}
// 체크포인트 결과 저장
checkpoints[checkpoint_idx] = *seed;
// 다음 체크포인트를 위한 솔트 증가
*salt = *salt + 1;
}
```

3. 솔트 관리 메커니즘

솔트는 각 체크포인트마다 증가하여 독립성을 보장합니다:

```
// 체크포인트 0: salt = base_salt + 0
// 체크포인트 1: salt = base_salt + 1
// 체크포인트 2: salt = base_salt + 2
// ...
```

솔트 계산 공식

```
pub const fn step_number_to_salt_number(config: &VdfConfig, step_number: u64)
match step_number {
0 => 0,
_ => (step_number - 1) * config.num_checkpoints_in_vdf_step as u64 + 1,
}
}
```

예시 계산:

- 스텝 0: 솔트 = 0
- 스텝 1: 솔트 = (1-1) × 25 + 1 = 1
- 스텝 2: 솔트 = (2-1) × 25 + 1 = 26
- 스텝 3: 솔트 = (3-1) × 25 + 1 = 51

체크포인트 검증 프로세스 (`last_step_checkpoints_is_valid`)

검증 함수 시그니처

```
pub async fn last_step_checkpoints_is_valid(
vdf_info: &VDFLimiterInfo, // 블록의 VDF 정보
config: &VdfConfig, // VDF 설정
) -> eyre::Result<()>
```

검증 단계별 분석

1. 입력 유효성 검사

```
// 글로벌 스텝 번호 검증
if vdf_info.global_step_number == 0 {
return Err(eyre::eyre!(
"Invalid global_step_number: must be greater than 0"
));
}

// 스텝과 체크포인트 일관성 검증
let last_step = vdf_info.steps.0.last()
.ok_or(eyre::eyre!("No steps are found in the block"))?;
let last_step_last_checkpoint = vdf_info.last_step_checkpoints.0.last()
.ok_or(eyre::eyre!("No last checkpoints are found in the block"))?;

if *last_step != vdf_info.output {
return Err(eyre::eyre!("The last step does not match the output"));
}
if last_step != last_step_last_checkpoint {
return Err(eyre::eyre!(
"The last step does not match the last checkpoint"
));
```

```
}
```

2. 시드 결정 로직

```
let mut seed = if vdf_info.steps.len() >= 2 {
// 이전 스텝의 출력을 시드로 사용
vdf_info.steps[vdf_info.steps.len() - 2]
} else {
// 첫 번째 스텝인 경우 이전 출력 사용
vdf_info.prev_output
};
```

3. 리셋 시드 적용

```
let global_step_number: usize = vdf_info.global_step_number.try_into()
.wrap_err("Should run in a 64 bits architecture!")?;

// 리셋 주기 확인 및 엔트로피 적용
if (global_step_number > 1) && ((global_step_number - 1) % config.reset_frequenc
tracing::info!(
"Applying reset step: {} seed {:?}",
global_step_number,
seed
);
```

```
seed = apply_reset_seed(seed, reset_seed);
} else {
tracing::info!(
"Not applying reset step: {} seed {:?}",
global_step_number,
seed
);
}
```

4. 병렬 검증 실행

```
let test = tokio::task::spawn_blocking(move || {
// 스레드 풀 생성 (설정된 제한 내에서)
let pool = rayon::ThreadPoolBuilder::new()
.num_threads(config.parallel_verification_thread_limit)
.build()
.unwrap();

let num_iterations = config.num_iterations_per_checkpoint();
// 병렬 체크포인트 계산
let test: Vec<H256> = pool.install(|| {
(0..config.num_checkpoints_in_vdf_step)
.into_par_iter() // 병렬 반복자
.map(|i| {
```

```
let mut salt_buff: [u8; 32] = [0; 32];
(start_salt + i).to_little_endian(&mut salt_buff);
let mut seed = cp[i];
let mut hasher = Sha256::new();

// 각 체크포인트 독립적으로 계산
for _ in 0..num_iterations {
hasher.update(salt_buff);
hasher.update(seed.as_bytes());
seed = H256(hasher.finalize_reset().into());
}
seed
})
.collect::<Vec<H256>>()
});
test
}).await?;
```

5. 결과 비교 및 검증

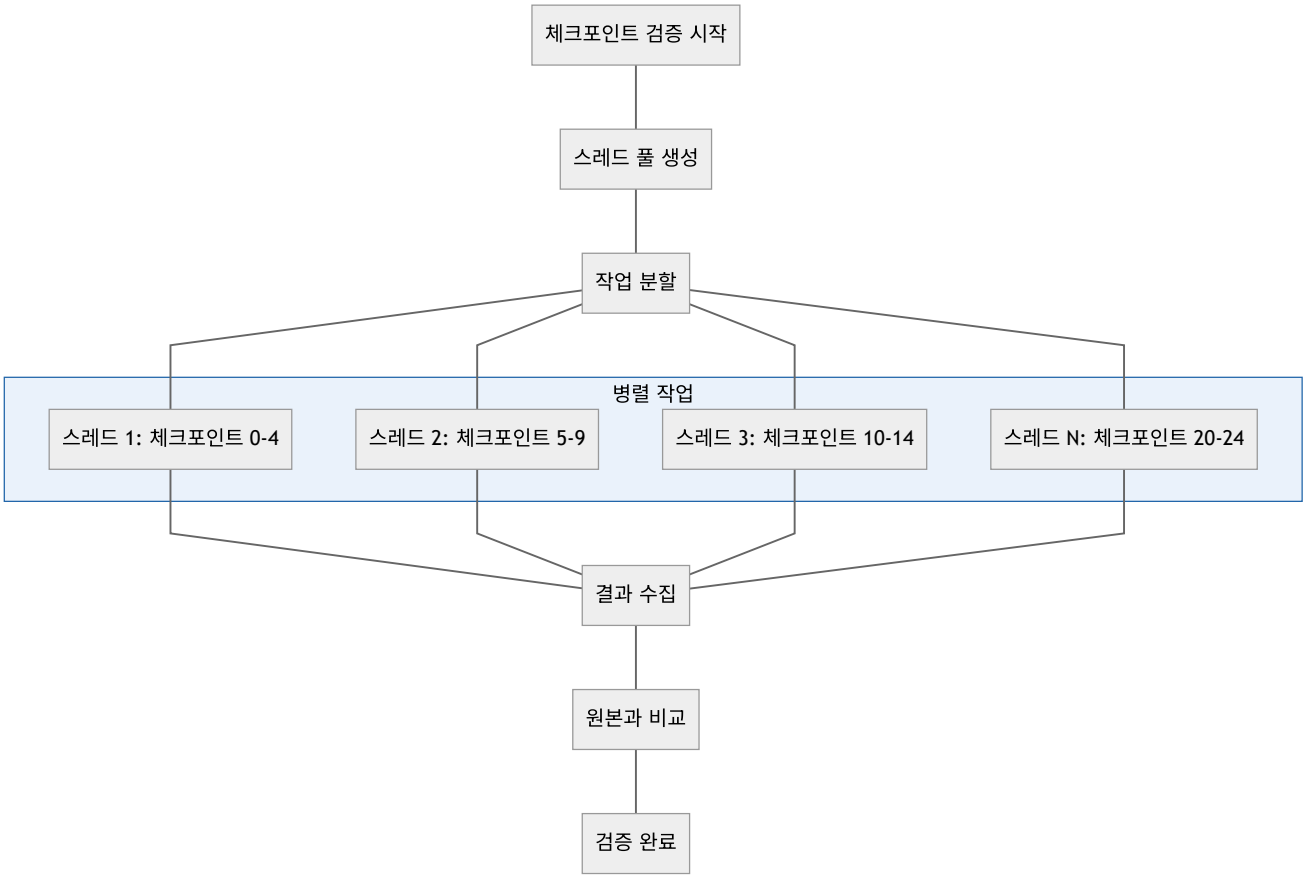
```
let is_valid = test == vdf_info.last_step_checkpoints;

if !is_valid {
```

```
// 불일치 항목 상세 로깅

warn_mismatches(&vdf_info.last_step_checkpoints, &H256List(test));

Err(eyre::eyre!("Checkpoints are invalid"))

} else {

Ok(())

}
```

병렬 검증 시스템

병렬화 전략

VDF 체크포인트 시스템의 핵심 장점은 검증 단계에서의 병렬 처리입니다:



성능 최적화 기법

1. 스레드 풀 관리

```
let  pool = rayon::ThreadPoolBuilder::new()
.num_threads(config.parallel_verification_thread_limit)
.build()
```

```
.unwrap();
```

- 설정 가능한 스레드 수 제한
- 시스템 과부하 방지
- CPU 코어 수에 맞춘 최적화

2. 작업 분할 전략

```
(0..config.num_checkpoints_in_vdf_step)
.into_par_iter() // 각 체크포인트를 독립적 작업으로 분할
.map(|i| {
// 독립적인 체크포인트 계산
})
```

3. 메모리 효율성

```
let mut salt_buff: [u8; 32] = [0; 32]; // 스택 할당
let mut hasher = Sha256::new(); // 경량 해시 인스턴스
```

성능 벤치마크

단일 스레드 vs 병렬 처리 비교:

```
설정: 25개 체크포인트, 40,000 반복/체크포인트

단일 스레드:
```

```
- 계산 시간: ~1000ms
- CPU 사용률: 25% (4코어 시스템)


병렬 처리 (4스레드):
- 계산 시간: ~250ms
- CPU 사용률: 100%
- 속도 향상: 4배
```

체크포인트 데이터 구조

VDFLimiterInfo 구조

```rust
pub struct VDFLimiterInfo {
pub output: H256, // 최종 VDF 출력
pub global_step_number: u64, // 글로벌 스텝 번호
pub seed: H256, // 리셋 시드
pub next_seed: H256, // 다음 리셋 시드
pub prev_output: H256, // 이전 출력
pub last_step_checkpoints: H256List, // 마지막 스텝 체크포인트들
pub steps: H256List, // 스텝 해시들
pub vdf_difficulty: Option<u64>, // VDF 난이도
pub next_vdf_difficulty: Option<u64>, // 다음 VDF 난이도
}
```

체크포인트 배열 구조

```
// 체크포인트는 H256 해시의 벡터로 저장
pub struct H256List(pub Vec<H256>);

// 각 체크포인트는 32바이트 해시
pub struct H256(pub [u8; 32]);
```

오류 처리 및 디버깅

불일치 감지 및 로깅

```
pub fn warn_mismatches(a: &H256List, b: &H256List) {
let mismatches: Vec<(usize, (&H256, &H256))> =
a.0.iter()
.zip(&(b.0))
.enumerate()
.filter(|(_i, (a, b))| a != b)
.collect();

for (index, (a, b)) in mismatches {
tracing::error!(
"Mismatched hashes at index {}: expected {:?} got {:?}",
```

```
index,
a,
b
);
}
}
```

일반적인 오류 상황

1. 체크포인트 불일치

원인:

- 잘못된 시드 값
- 솔트 계산 오류
- 반복 횟수 불일치

해결 방법:

```
// 상세 로그 활성화
tracing::debug!("VDF verification details: seed={:?}, salt={:?}", seed, salt);

// 단계별 검증
assert_eq!(computed_checkpoint, expected_checkpoint,
"Checkpoint {} mismatch", checkpoint_index);
```

2. 병렬 처리 오류

원인:

- 스레드 풀 생성 실패
- 메모리 부족
- 동시성 문제

해결 방법:

```
// 스레드 수 동적 조정
let num_threads = std::cmp::min(
config.parallel_verification_thread_limit,
num_cpus::get()
);
```

최적화 전략

1. 캐시 활용

```
// 자주 사용되는 솔트 값 캐싱
static SALT_CACHE: Lazy<HashMap<u64, [u8; 32]>> = Lazy::new(HashMap::new);
```

2. 메모리 풀링

```
// 해시 인스턴스 재사용
thread_local! {
static HASHER_POOL: RefCell<Vec<Sha256>> = RefCell::new(Vec::new());
```

```
}
```

3. SIMD 최적화

```
// 가능한 경우 SIMD 명령어 활용
#[cfg(target_feature = "sha")]
use std::arch::x86_64::*;
```

보안 고려사항

1. 타이밍 공격 방지

```
// 상수 시간 비교 사용
use subtle::ConstantTimeEq;

if  computed_hash.ct_eq(&expected_hash).into() {
// 검증 성공
}
```

2. 메모리 보안

```
// 민감한 데이터 제로화
use zeroize::Zeroize;
```

```rust
impl Drop for CheckpointData {
fn drop(&mut self) {
self.seed.zeroize();
self.salt.zeroize();
}
}
```

테스트 및 검증

단위 테스트 예시

```rust
#[tokio::test]
async fn test_checkpoint_validation() {
let config = VdfConfig::testing();
let vdf_info = create_test_vdf_info();
let result = last_step_checkpoints_is_valid(&vdf_info, &config).await;
assert!(result.is_ok(), "Valid checkpoints should pass verification");
}

#[test]
fn test_parallel_vs_sequential() {
let checkpoints_parallel = calculate_checkpoints_parallel();
```

```
let checkpoints_sequential = calculate_checkpoints_sequential();
assert_eq!(checkpoints_parallel, checkpoints_sequential,
"Parallel and sequential results must match");
}
```

성능 테스트

```
#[bench]
fn bench_checkpoint_verification(b: &mut Bencher) {
let vdf_info = create_large_vdf_info();
let config = VdfConfig::default();
b.iter(|| {
black_box(last_step_checkpoints_is_valid(&vdf_info, &config));
});
}
```

다음 단계

체크포인트 시스템의 이해를 바탕으로 다음 문서에서는 VDF의 난이도 조정 및 리셋 메커니즘을 다룹니다:

- 난이도 조정 메커니즘 - 동적 난이도 조정 및 VDF 리셋 시스템
- VDF 성능 최적화 - 고급 최적화 기법 및 하드웨어 가속